Name: Lawrence Oro

Course: CPSC 39

Date: 24 – April – 2026

Program Overview

The Library Management System is a Java-based application designed to optimize productivity by helping users organize their personal book collections. It solves the problem of having to manually/mentally track books you own, are currently reading, and or have read. By providing the user with a persistent, searchable, and maintainable database of each book's details, such as ISBN, Genre, and Reading Status, we take the mental burden off the user.

Data Structures Implemented

For the following program, I decided to utilize the following data structures to manage data efficiently. An **ArrayList** (in *myLibrary*) was used to store *Book* objects in memory. I chose to use this structure due to its dynamic resizing capabilities and, more importantly, its time complexity of **O (1)**, which was necessary for the indexing used in my sorting and searching algorithms (listed below)

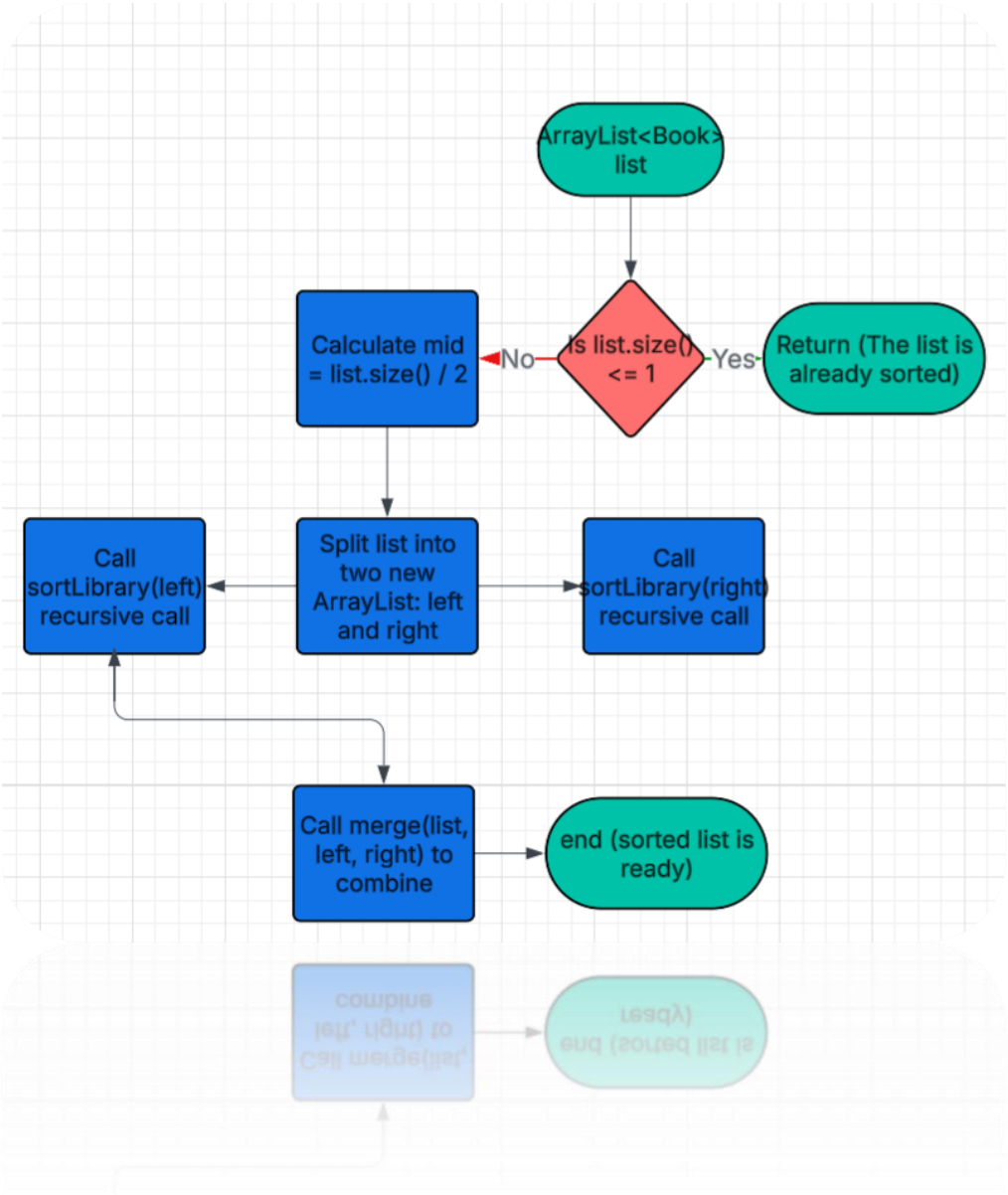
Additionally, I implemented the **Stack** data structure (in *undoStack*). This was used to design an 'Undo' feature for the user for easy retrieval of deleted data. I decided to use this data structure because it follows the **Last-In-First-Out (LIFO)** logic, which ensures that the most recently deleted book is the first one to be restored back into the database.

Now, lastly, I used the **Arrays** data structure. This was used for data parsing and specifically, the (.split(“,”)) method, which is used to parse CSV data from my local books.txt file into the individual object attributes before being stored in the users' collections. I also used String extensively to compare titles and data encapsulation.

Algorithms Implemented

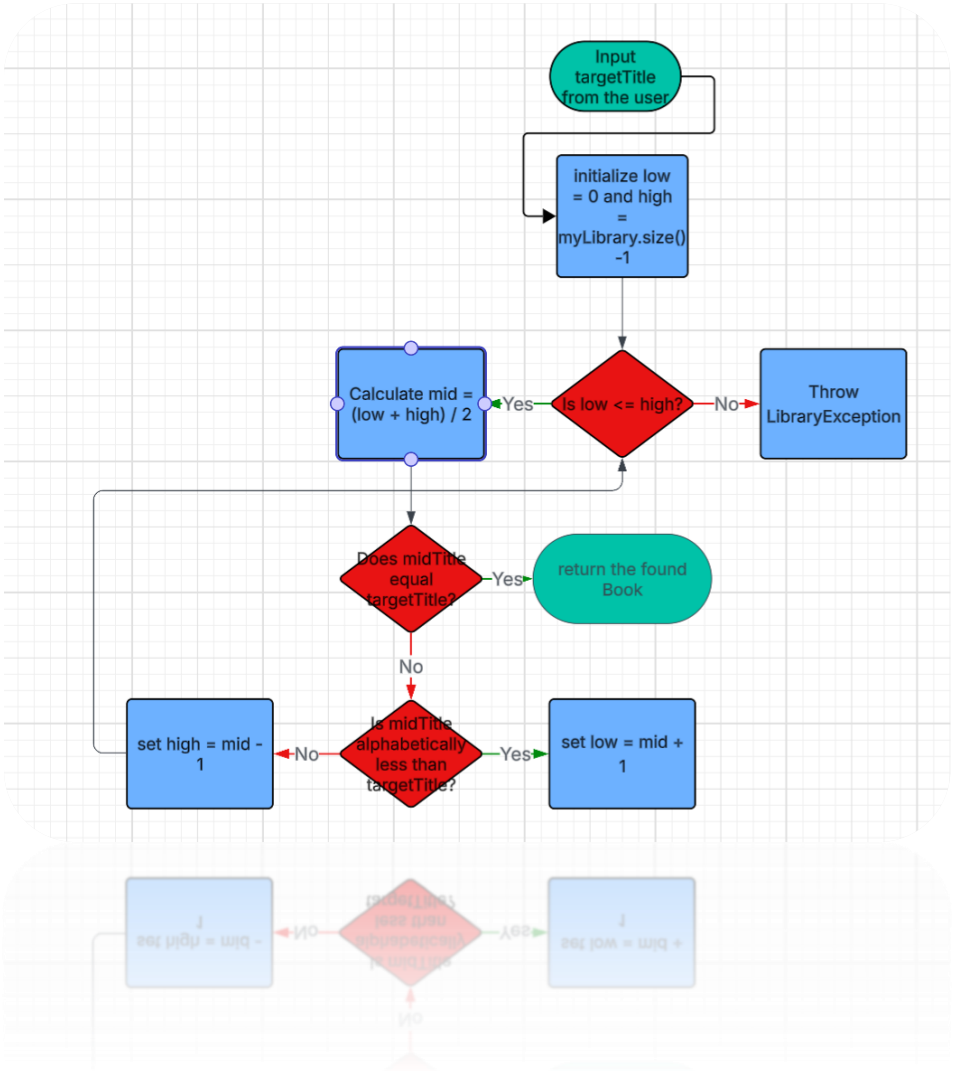
The first algorithm that I decided to use to effectively search through the users' database was the **Merge sort** algorithm. Why, may you ask? This will automatically sort the library

alphabetically by title before the library is displayed to the user. Due to Its recursive beauty, the merge sort algorithm will yield a time complexity of O (n log n). This logarithmic factor is vital to the program because it ensures that sorting remains fast and efficient even as the user's list of books grows exponentially large. The steps to the merge sort algorithm are: (flowchart below)



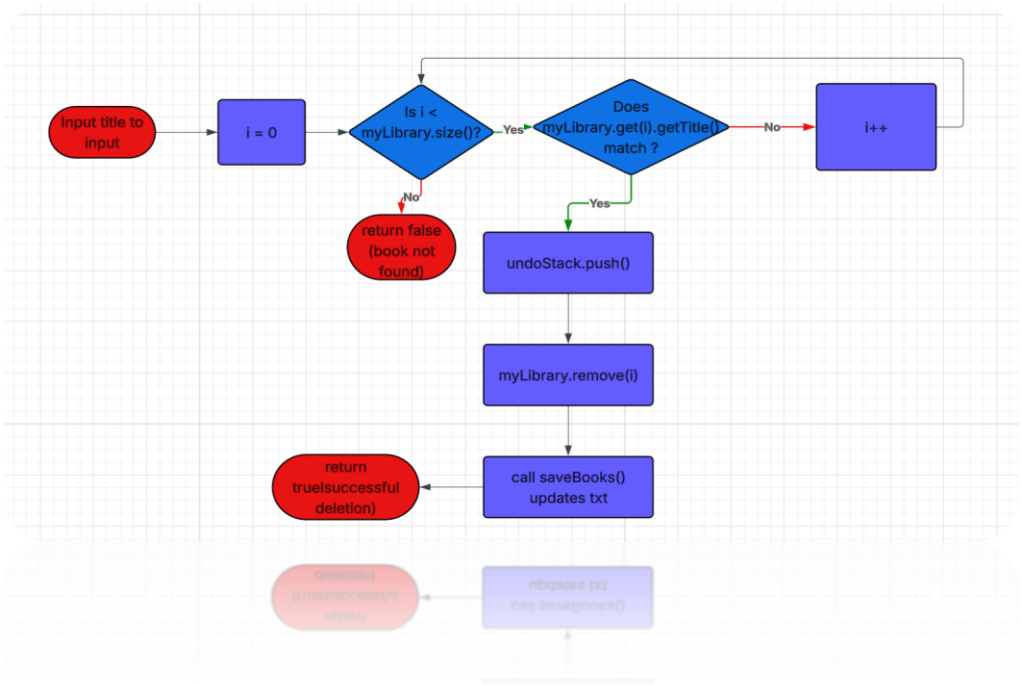
Binary Search was then the next algorithm I decided to implement into this library management system to efficiently locate and return a book by its title to the user. I chose this algorithm due to its complexity of *O (log n)*. This is crucial because when the users' collection becomes much larger, the program will still run smoothly without delay by cutting the search area in half with every iteration. The steps to the binary search algorithm

are: (flowchart below).



To finish off, the last algorithm that is used inside this program is **Linear Search**. The purpose of using a linear search was to find a specific title to remove from the user's collection. Although not highly effective for bigger databases for this initial version, considering the size of the users' collection, it was most efficient to use a Linear Search, which yields a time complexity of *O (n)*. This means that the time taken to find a book grows linearly with the number of books in the collection. The steps to the Linear Search

algorithm are as follows: (flowchart below).



Development Reflection

In the process of developing this project, I ran into many issues when it came to **logic/abstraction** and **output design**. The first problem I ran into was with the **Scanner Buffer**. I encountered a bug where the program would skip the 'Genre' input. I discovered that using the *nextLong()* Java method for the *ISBN* left a newline character inside the buffer. I knew how to fix this situation; I had to add an extra *input.nextLine()* to clear the buffer. I later ran my program and saw that anytime I input an ISBN number to add to the collection, I would get returned my error message stating that I need to input integer values only. I then realized it was better to have the ISBN read as a string rather than a long and trying to parse, because then this would allow for better data integrity. In doing this, I not only solved my buffer issue, but it also provided the user with a better/smoother input process

Later, once I had dealt with some of the backend logic, I decided to move my focus onto the main menu that would be displayed in the terminal. Originally, all my menu text was set inside of Main.java. I improved this design by creating the MenuConstants.java class. In doing this, it centralized the UI strings, added an extra layer of abstraction to my program, and most importantly, this makes the code much more maintainable for any future development/versions.

To further add to the abstraction of my project, I noticed that returning null for a missing book in the collection was a poor design. After thinking about it and doing some documentation research, and asking some questions to my LLM, I created a custom LibraryException class to handle missing books from the collection that the user may be trying to find. This allowed me to then use the try – catch blocks in the UI for a more professional or aesthetically pleasing UX.

Version 2.0 Plans

In the next version of the Library Management System, I will be looking into transitioning this program from a terminal-based app to a windowed application (GUI). Furthermore, for more robust data management, I can replace the local .txt file with an SQL database integration to allow for more complex data relationships and sorting. Finally, to make for better searching, I could do some API connection using a book API to automatically fetch author and genre details for the user when they enter an ISBN #.